

Report (MP-3)

Distributed Sorting System Performance

1. Implementation Analysis

Merge sort

Multiple threads are created to sort chunks of data in parallel (sorting phase) and finally these sorted threads are merged sequentially (merging phase) by directly using the main thread.

Sorting Phase (Parallel): Firstly the num of threads to be executed are calculated. Each thread independently sorts a separate chunk of the array. This allows each thread to operate independently as each thread has exclusive access to the chunk. Sorting different parts at the same time reduces the overall sorting time compared to sequential sorting.

Merging Phase (Sequential): After each chunk is sorted, the main thread sequentially merges pairs of sorted chunks until a single sorted array remains. The merging is done iteratively, doubling the size of merged array in each step.

Pros and Cons

Pros:

1. Efficiency/Performance: By using multiple threads for sorting, we were able to sort big chunks of the array independently and parallelly.
2. Reduced Complexity in Merging: Performing the merging in a single-threaded fashion simplified the management of shared data and memory by preventing memory conflict issues which would have arisen if each thread was merging independently.
3. Memory Efficiency: Minimises memory usage because temporary arrays for merging are only created when necessary, and the main thread handles them iteratively, releasing memory once each merge step is complete.

Cons:

1. Limited Scalability: Since merging doesn't use multiple threads, for very large datasets this may take significantly longer than the sorting phase increasing the overall time complexity of the algorithm.

2. Not fixed array Lengths: As maximum number of threads are fixed, the chunk size differs for every input. As the size of criterion on the basis of which sorting has to be performed grows large, the chunk size grows leading to increase in time and memory complexity.

Count sort

The counting task of updating the count array is divided into multiple threads, with each thread responsible for counting occurrences within a certain specified start and end index. Global counts are updated by each thread using locks to prevent race conditions. Finally elements are placed in the output array based on cumulative counts.

Counting Phase (Parallel):

Each thread is assigned a segment (using start and end index). It then independently counts occurrences of each key within its range. This helps increase efficiency as the counting process can be carried out parallelly rather than sequentially reducing the time spent. After this, threads synchronise and use a mutex lock to update a global count array.

Cumulative Count and Placement Phase (Parallel with Synchronisation):

Cumulative sum is calculated for all elements of the count array using threads. Each thread calculates where elements from its segment should be placed in the output array.

Synchronisation is carried out using locks as each thread needs to position its elements based on cumulative counts derived from the global count array.

Pros and Cons

Pros:

1. Efficiency/ Performance: By splitting the counting task to multiple threads, we are able to count and calculate the cumulative sum independently and parallelly.
2. Minimal Synchronisation Overhead: Using local counts reduces the frequency of mutex locks, as each thread only updates the global count array once it completes its counting individually
3. Memory Efficiency: Each thread uses a small local_counts array that is deallocated after updating the global count. This avoids large memory overhead, as the global counts array only needs to be updated once per key range.

Cons:

1. Counting sort is optimal for a limited key range. If the key range is very large or data is sparse, the counting sort approach becomes inefficient in terms of memory usage. For example: When sorting using the name or timestamp category, the key value to be sorted becomes very large which goes beyond the range for counting sort to work.

2. Execution Time Analysis

Data for execution and memory taken from log file.

Count Sort

Data Size Impact:

- Execution times increase with larger data sizes, particularly for sorting by Name. Sorting by Name takes progressively longer as dataset size grows, from **0.019 seconds** for size 2 to **0.045 seconds** for size 41. Sorting by ID and timestamp involve simple unique mapping, fitting the typical runtime of $O(n+k)$.

Sorting Criterion Impact:

- Sorting by ID and Timestamp is significantly faster than sorting by Name. This is evident even in smaller datasets (e.g., size 2, where ID sorting takes **0.009 seconds** and Name sorting takes **0.019 seconds**). Sorting by Timestamp is the fastest across most sizes. Sorting by Name requires extra memory and processing time due to key generation from strings unlike direct key assignment in ID.

Scalability:

- Sorting by ID and Timestamp scales well, showing only minor increases in time as dataset size grows. ID sorting remains below **0.02 seconds** up to size 41. Sorting by Timestamp remains efficient due to its unique key assignment per timestamp.
- Sorting by Name is the least scalable due to managing the larger counts array.

Merge Sort

Data Size Impact:

- Execution times generally increase with larger datasets in line with the typical runtime of merge sort, $O(n \log n)$. Sorting by ID takes **0.0049 seconds** for a dataset

of 45 entries, but **0.107 seconds** for 63,685 entries and **0.218 seconds** for the largest dataset of 95,528 entries.

Sorting Criterion Impact:

- Across dataset sizes, the sorting criterion (ID, Name, or Timestamp) results in only minor variations in execution time. With 95,528 entries, sorting by ID, Name, and Timestamp took **0.1079**, **0.1618**, and **0.2183** seconds, respectively.
- The differences here are due to the complexity of comparing each type of data: ID (numeric comparisons) is the fastest, while Name (string comparisons) and Timestamp (ISO 8601 format comparisons) take slightly longer.

Scalability:

- For larger datasets, the overhead of thread management and synchronization during merging begins to add to execution time, which could limit scalability gains.
- Distributed Merge Sort demonstrates strong scalability for both ID and Timestamp sorting, with execution times growing predictably with dataset size. Name could show more significant time increases as dataset and Name size grows.

3. Memory Usage Overview

Merge sort

Overall space complexity of the algorithm is $O(n)$.

As the algorithm uses linear space complexity, the memory required grows linearly with increasing number of file (n). Analysing for each sorting criterion separately we observe: Memory usage remains relatively low and stable across dataset sizes.

Count sort

Overall space Complexity: $O(n + k)$. As the algorithm uses linear space complexity, the memory required grows linearly with increasing number of file (n). Analysing for each sorting criterion separately we observe: Sorting by ID and Timestamp: Memory usage remains relatively low and stable across dataset sizes. Sorting by name: requires significantly more memory due to the overhead of storing keys for alphabetic strings(string_to_key) and using larger arrays to manage counts and cumulative placements.

File count

1. SMALL (15 files): Algorithm used is count_sort

Memory used: About 7000 KB

2. MEDIUM (100 files): Algorithm used is merge_sort

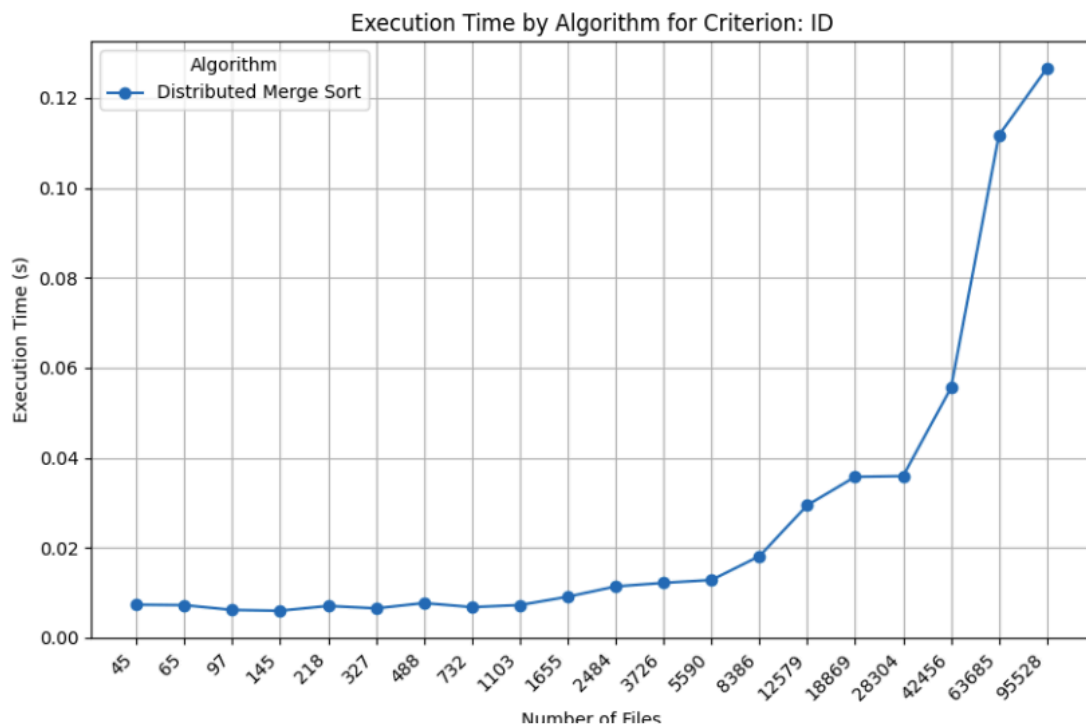
Memory used: About 2000 KB

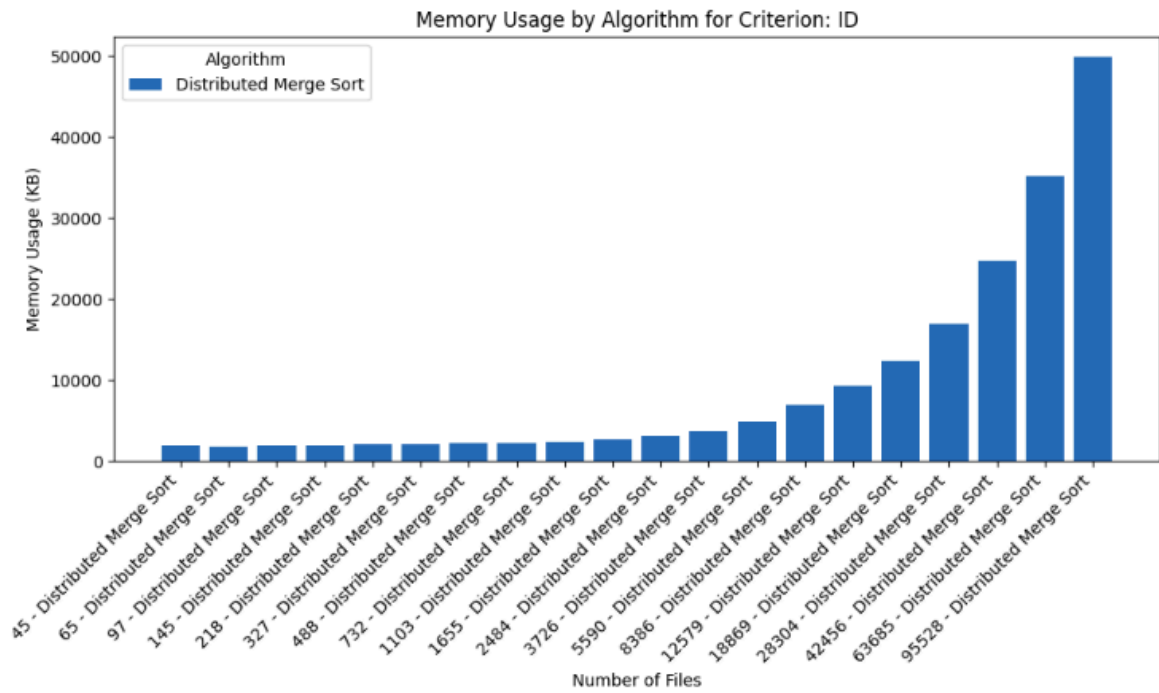
3. HIGH (10000 files): Algorithm used is merge_sort

Memory used: About 10000 KB

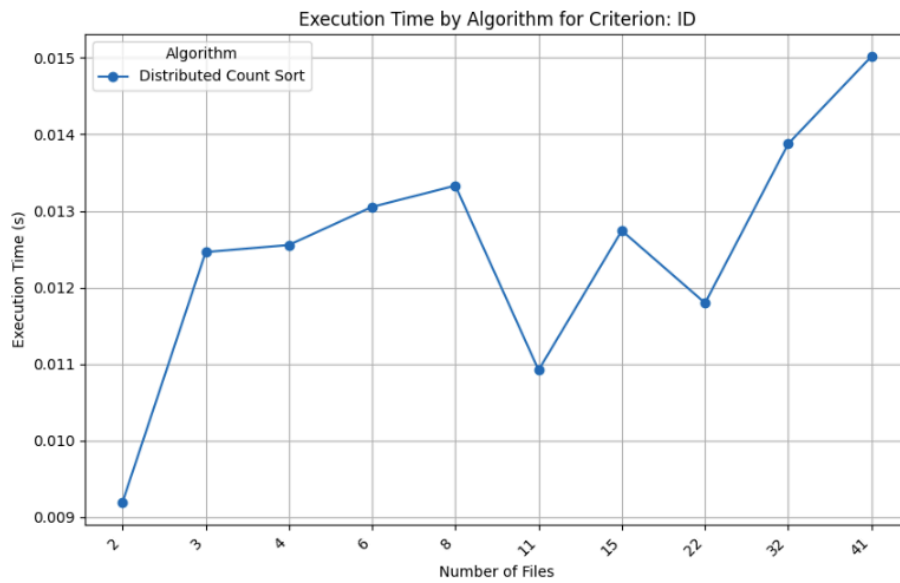
4. Graphs

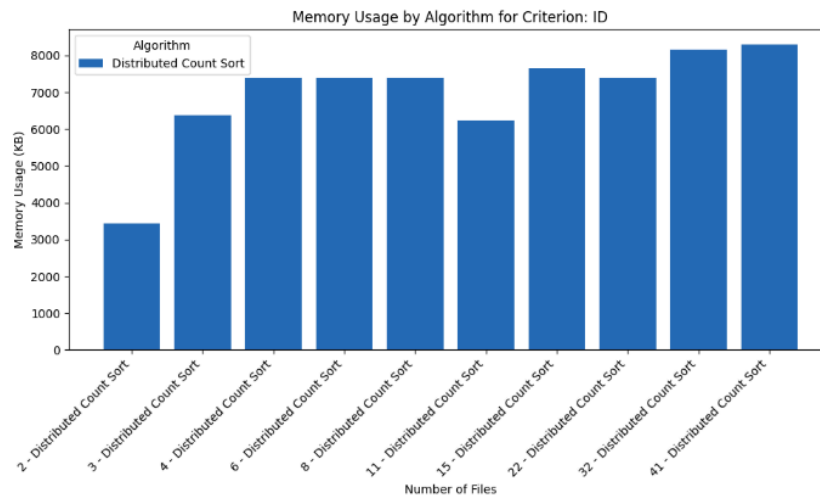
Merge sort



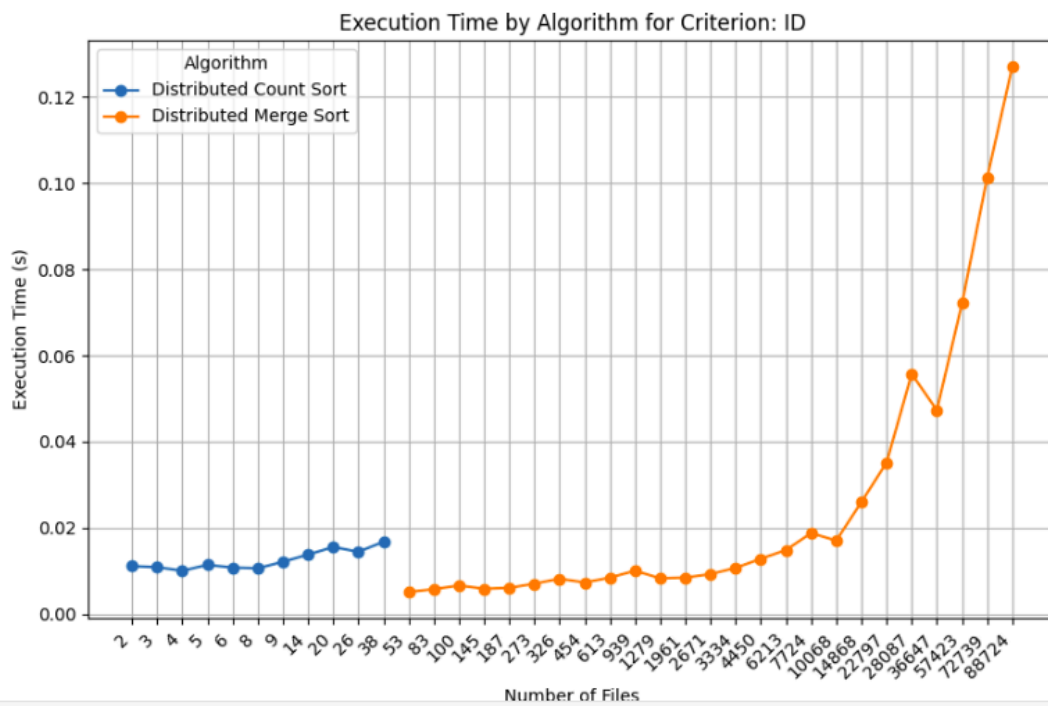


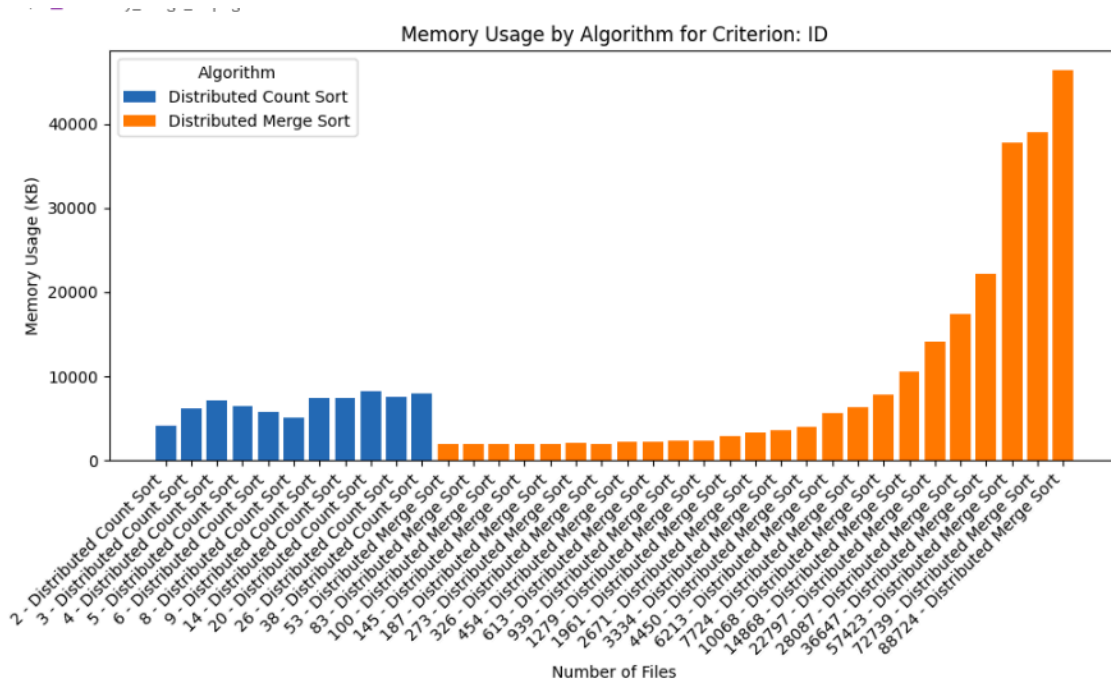
Count sort





Comparison graphs





5. Summary

Count Sort:

- Performs exceptionally well with datasets where the range (max-min) is relatively small compared to the number of files (n). Time complexity of $O(n + k)$, so time increases linearly with increase in number of files. Execution time is usually in range of **0.01 to 0.02 sec**.
- Requires $O(n + k)$ space. Memory consumption can increase significantly if the key range is large, especially when sorting based on attributes like timestamps or large numerical IDs. Memory usually in range of **5000-10000 KB**
- **Best Use Case:** Optimal for sorting files when the key range is limited

Merge Sort

- Consistent time complexity of $O(n \log n)$ for all datasets. Handles large datasets efficiently, regardless of key distribution. Execution time is usually in range of **0.005 to 0.12 sec** (for very large num of files)
- Uses additional space of $O(n)$ due to need for temporary arrays during the merge process. Only depends on number of files. Memory usually in range of **2000-40000 KB** (for very large num of files).
- **Best Use Case:** Suitable for scenarios where stability is important, or the key range is vast, and Counting Sort would be less efficient due to its higher memory demands.

Potential Optimizations for Larger Datasets

Count Sort: Using a hash map or sparse array structure instead of a dense array for counting, especially when the key range is large but sparsely populated. A slight deviation of count sort which is the radix sort can also be used. If a single hash is not required for sorting, for timestamps, count sort can be applied individually for sec then min then hrs and then date so on.

Merge Sort: Multi-threading to be used for both splitting the data set and sorting individually and the merging the threads. This parallelisation will improve efficiency and performance. It will require proper use of thread synchronisation and locks to prevent race conditions.

Analysis of Copy on Write Fork in XV6:

1.Counting Number of Page Faults

Output from lazytest:

```
$ lazytest
simple: Process 4 had 0 COW faults
ok
simple: Process 5 had 0 COW faults
ok
three: Process 6 had 4096 COW faults
Process 7 had 6554 COW faults
ok
three: Process 8 had 4096 COW faults
Process 9 had 6554 COW faults
ok
three: Process 10 had 4096 COW faults
Process 11 had 6554 COW faults
ok
file: Process 13 had 0 COW faults
Process 14 had 0 COW faults
Process 12 had 0 COW faults
Process 15 had 0 COW faults
ok
ALL COW TESTS PASSED
Process 3 had 24588 COW faults
```

Analysis of output

simpletest

This test allocates a large amount of memory and forks a child. The child process immediately exits without modifying any memory.

Expected Page Faults: Since the child does not write to any shared memory pages, no COW page faults are expected.

Observed COW Page Faults:

Process 4 and Process 5 both had **0 COW faults**.

This matches the expected behaviour, confirming that read-only memory access after fork does not trigger COW faults. The system correctly shares pages without duplication, showing that COW efficiently avoids unnecessary page allocation when pages are only read.

threetest

This test creates a hierarchy of processes (parent, child, and grandchild) where each process writes to different portions of a shared memory.

Expected Page Faults:

Each process writes to a shared page, which should result in multiple COW faults—each write operation to a previously shared page requires a new page allocation.

Observed COW Page Faults:

The observed COW page fault counts reflect the cumulative effect of each process's modifications.

filetest

This test examines COW behaviour during inter-process communication via pipes. filetest writes data into a buffer, which is then read by child processes. No process writes directly to shared pages.

Child Process:

Reads from the pipe and compares the read value to the expected value.

Does not write to buf or any other shared memory with the parent.

Parent Process:

Writes an integer to the pipe for the child to read.

Verification: After all child processes exit, the parent verifies that `buff[0]` is still 99, confirming that no child modified buffer

Expected COW Faults: 0

Each child process only reads from the pipe and does not perform any writes to shared memory.

No write operations on buffer or other shared pages mean no COW faults are triggered.

Observed COW Page Faults

Process 12, Process 13, Process 14, and Process 15 all had 0 COW faults

2. Efficiency Analysis

Benefits of COW Implementation

1. Memory Conservation

COW fork optimizes memory usage by sharing pages between the parent and child processes after fork. Pages are duplicated only when a process modifies them, significantly reducing memory consumption, especially for processes that perform minimal modifications.

This approach is particularly efficient for processes that mostly read shared data, as `simpletest` and `filetest` demonstrate.

2. Performance Optimization

By avoiding immediate page duplication, the COW mechanism minimizes the workload during fork, allowing processes to be created quickly. This deferred allocation approach enables the system to handle large memory allocations (as in `simpletest`) without consuming excessive physical memory.

`threetest` illustrates that even with multiple processes modifying memory, only necessary pages are duplicated, showing that COW limits the overhead of page copying.

3. Potential Areas for Optimization

- 1. Selective Pre-Copying for Frequent Modifications:** In scenarios where processes are highly likely to modify a large portion of shared memory (as in `threetest`), selectively pre-copying certain pages in advance could reduce the overhead of handling individual COW faults. For example, if an entire region

is known to be frequently modified, pre-allocating copies for that region could reduce the cost of handling repeated COW faults.

2. **Optimising COW at Sub-Page-Level:** Currently, COW operates at the page level, meaning any write to a shared page triggers a full page copy. For small or sparse writes, implementing a finer-grained COW approach, such as sub-page-level copy-on-write, could improve memory efficiency and reduce unnecessary copying.